

Module 14 - chapter 9

Backup EC2 Volumes: Automate creating Snapshots

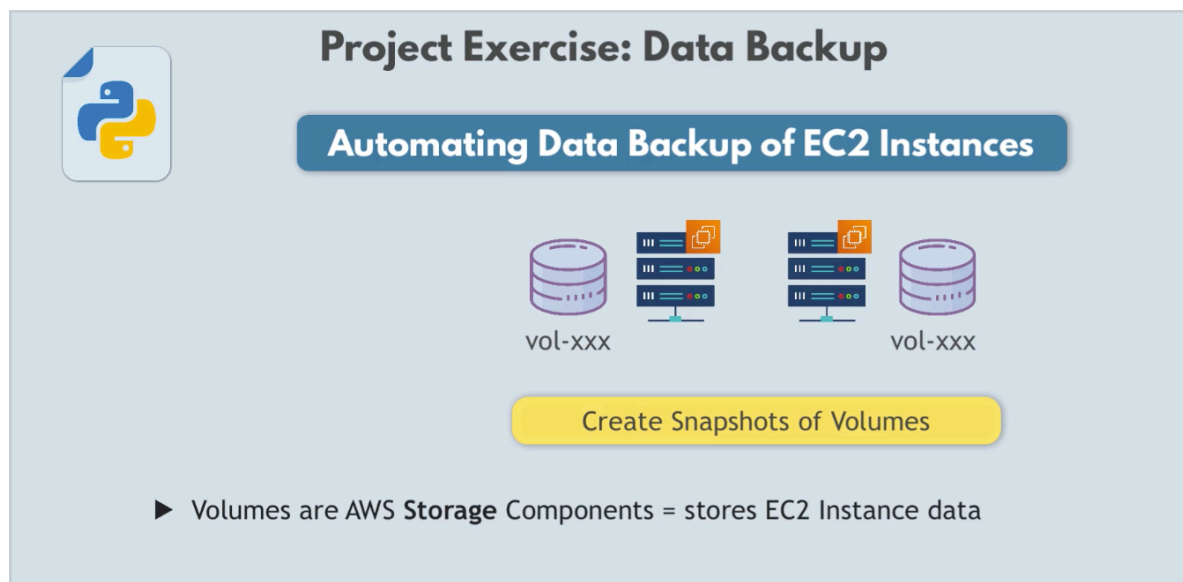
The project file used in this lecture can be found here:

- <https://gitlab.com/twn-devops-bootcamp/latest/14-automation-with-python/automation-projects/-/blob/main/volume-backups.py>

Data Backup

Project intro

Automate Data Backup for EC2 instances -> we will snapshots of Volumes for EC2 instances

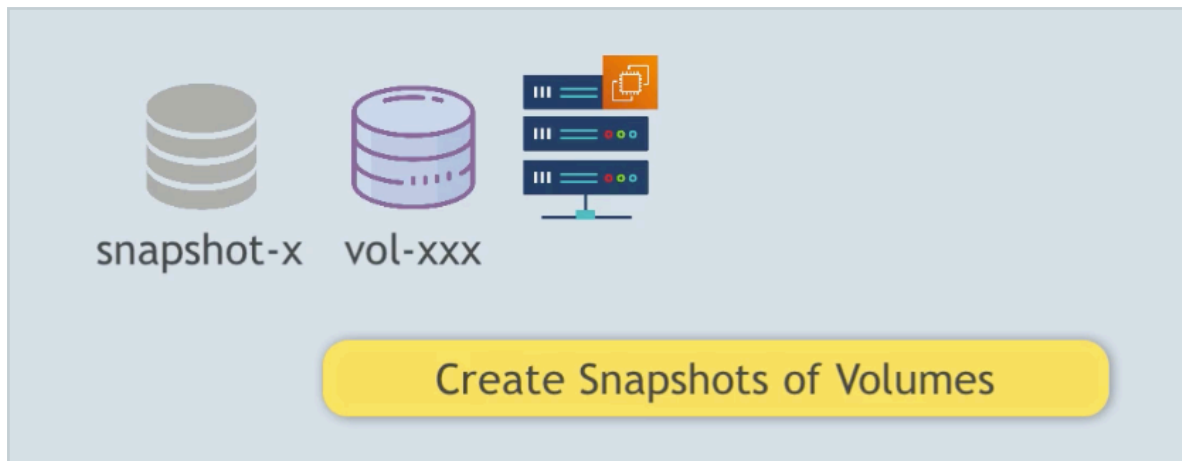


Volume -> A volume of an EC2 instance is like a hard drive for the EC2 server.

So when we create an EC2 instance a volume gets created automatically and when we delete an EC2 instance the volume gets deleted as well.

Snapshot -> a copy

Snapshot of volume -> copy of a volume at the moment of taking the snapshot.



 **Project Exercise: Data Backup**

Automating Data Backup of EC2 Instances

    Backup and Recovery

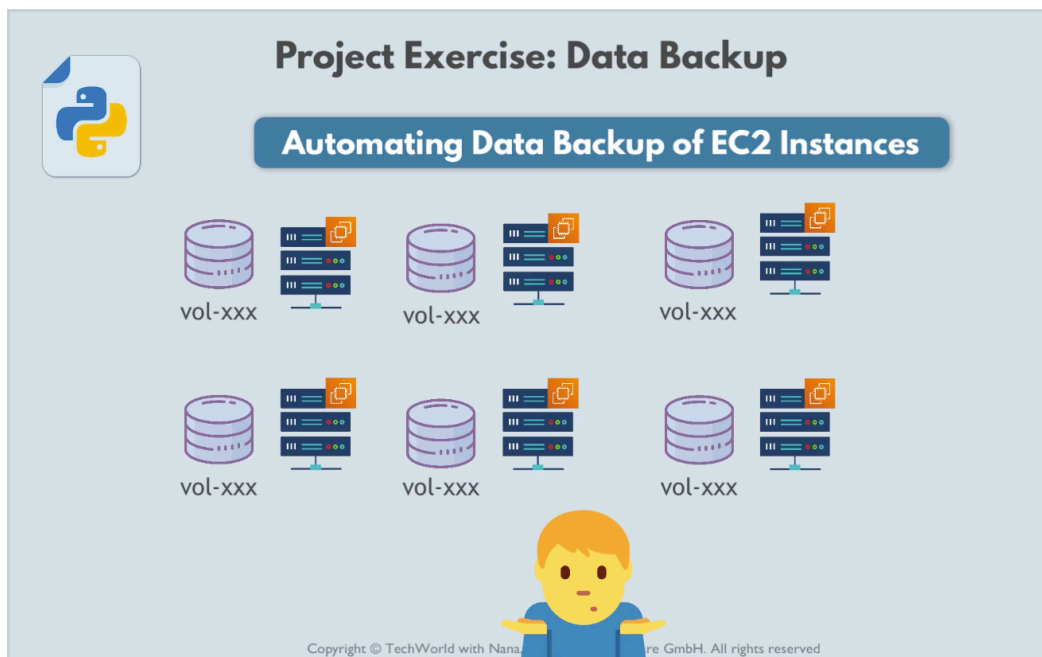
Create Snapshots of Volumes

- ▶ Volumes are **AWS Storage Components** = stores EC2 Instance data
- ▶ Volume Snapshot = **copy** of Volume

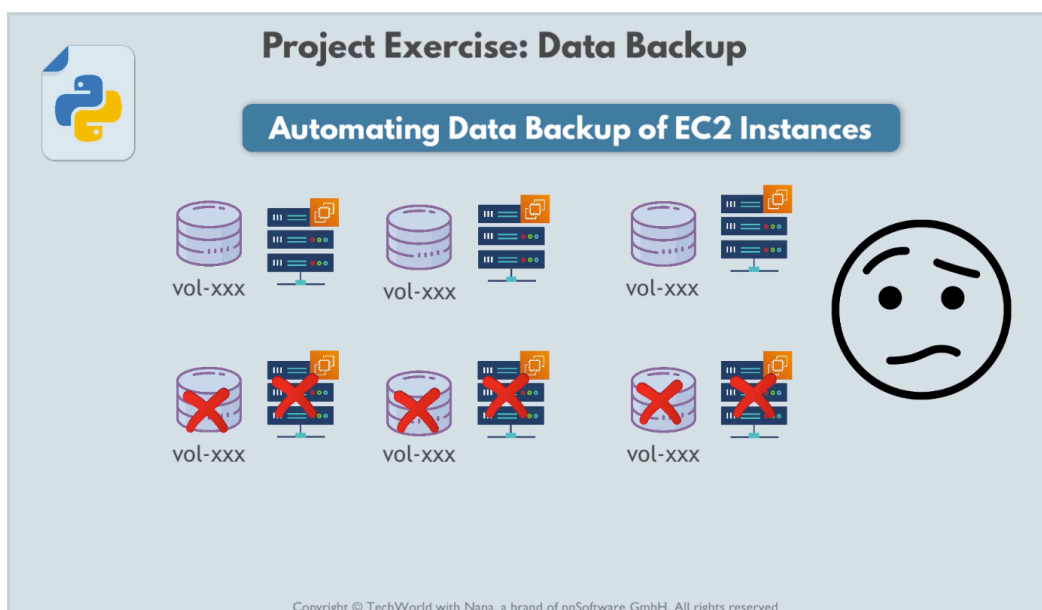
Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

Making snapshots and storing them needs to be automated to make sure that we have backups.

Now if we have 50 EC2 volumes that have not been backedUp



And then some EC2 instances crashed and we lost the data



So now the team thinks is important to make daily backups to avoid this in the future.

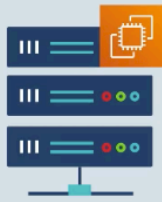
And now is our job to do the backups and we will automate the creation of the backups so we don't have to do it ourselves every single day.

We will make a python program to run the backups every day at a specific time.

Create 2 EC2 instances

We will launch 2 servers manually one with tag dev and the other with tag prod

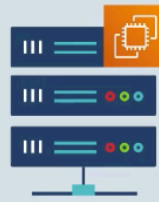
What we'll do



DEV



vol-xxx



PROD



vol-xxx

EC2 > Instances > Launch an instance

Additional costs apply for AMIs with pre-installed software

Key pair (login) Info
You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.
Key pair name - **required**
myapp-key-pair [Create new key pair](#)

Network settings Info [Edit](#)
Network Info
vpc-03677c8bf7cc2f715
Subnet Info
No preference (Default subnet in any availability zone)
Auto-assign public IP Info
Enable
Additional charges apply when outside of free tier allowance
Firewall (security groups) Info
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.
☐ Create security group ☒ Select existing security group
Common security groups Info
Select security groups
default: sg-0638f0ba242d48a96 [X](#)
vpc-03677c8bf7cc2f715
Security groups that you add or remove here will be added to or removed from all your network interfaces. [Compare security group rules](#)

Summary
Number of instances Info
2
When launching more than 1 instance, consider EC2 Auto Scaling
Software image (AMI)
Amazon Linux 2023 AMI 2023.12...[read more](#)
ami-0e9c70d9f136197bcb
Virtual server type (instance type)
t3.micro
Firewall (security group)
default
Storage (volumes)
1 volume(s) - 8 GiB
[Cancel](#) [Launch instance](#) [Preview code](#)

EC2 > Instances > Launch an instance

Success
Successfully initiated launch of Instances (i-09d1e327b01bb12ec, i-08cf2b587fe1ccd0b)

► **Launch log**

And we add the 'dev' tag

EC2 > Instances > i-09d1e327b01bb12ec > Manage tags

Manage tags Info
A tag is a custom label that you assign to an AWS resource. You can use tags to help organize and identify your instances.

Key Optional
Q Name [X](#) **Value - optional**
Q dev [X](#) [Remove](#)

[Add new tag](#)
You can add up to 49 more tags.

[Cancel](#) [Save](#)

Instances (1/2) Info

Last updated 2 minutes ago

Connect

Instance state

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

Instance state = running

Clear filters

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
☒	dev	i-09d1e327b01bb12ec	Running	t3.micro	Initializing	eu-west-2c	ec2-16-60-231-50.eu-w...	16.60.231.50
☐		i-08cf2b587fe1ccd0b	Running	t3.micro	Initializing	eu-west-2c	ec2-18-134-8-94.eu-we...	18.134.8.94

i-09d1e327b01bb12ec (dev)

And the 'prod' tag

Instances (1/2) Info

Last updated 2 minutes ago

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

Instance state = running

Clear filters

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☐	dev	i-09d1e327b01bb12ec	Running	t3.micro	Initializing	eu-west-2c	ec2-16-60-231-50.eu-w...
☒		i-08cf2b587fe1ccd0b	Running	t3.micro	Initializing	eu-west-2c	ec2-18-134-8-94.eu-we...

i-08cf2b587fe1ccd0b

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

Tags

Find tag

Key	Value
-----	-------

No tags found

Manage tags

EC2 > Instances > i-08cf2b587fe1ccd0b > Manage tags

Manage tags Info

A tag is a custom label that you assign to an AWS resource. You can use tags to help organize and identify your instances.

Key

Value - optional

Find Name

Find prod

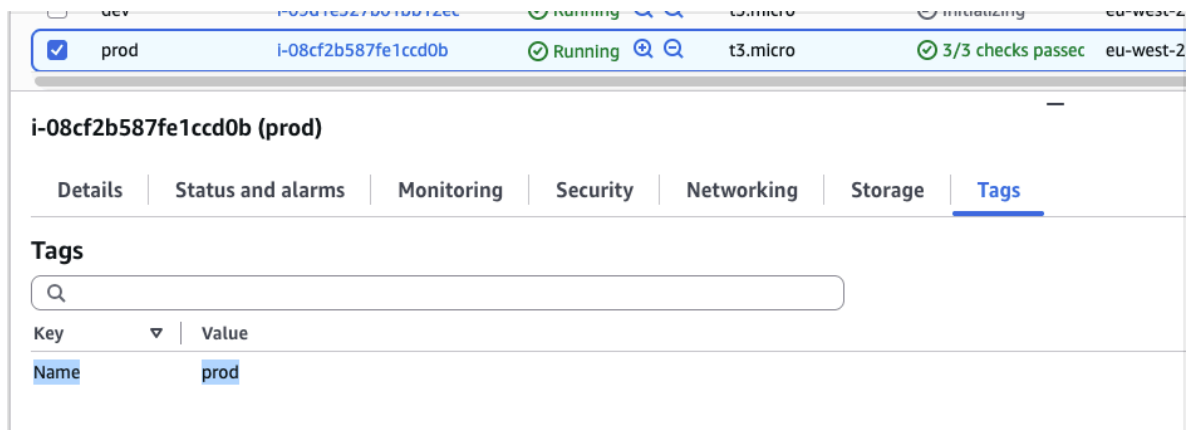
Remove

Add new tag

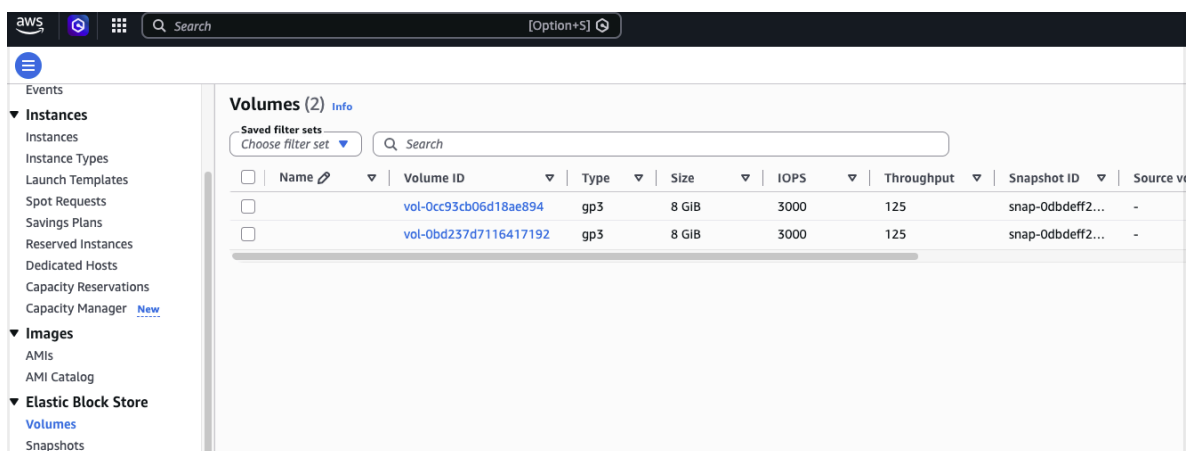
You can add up to 49 more tags.

Cancel

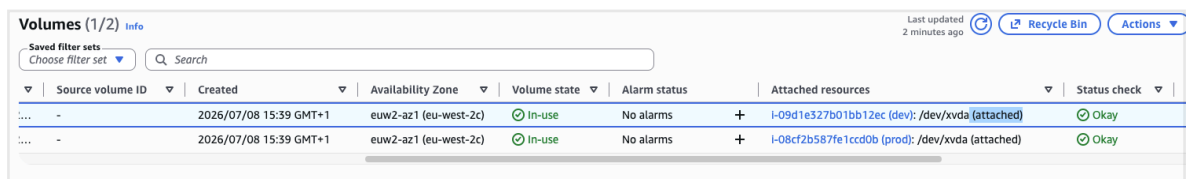
Save



Now let's go to volumes

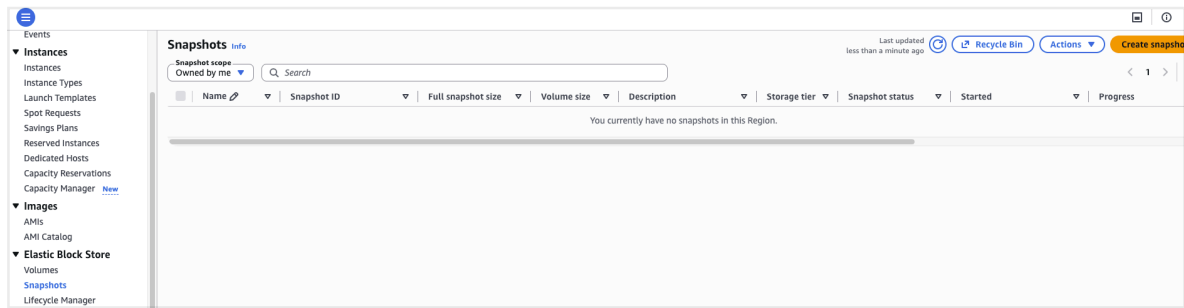


We can see two volumes, one for each EC2 server and here we can see the values are attached to the EC2 instances



And each volume have a size of 8GiB and they are of type gp3 -> think of them as hard drives attached to each server.

Now let's look at snapshot and it is empty

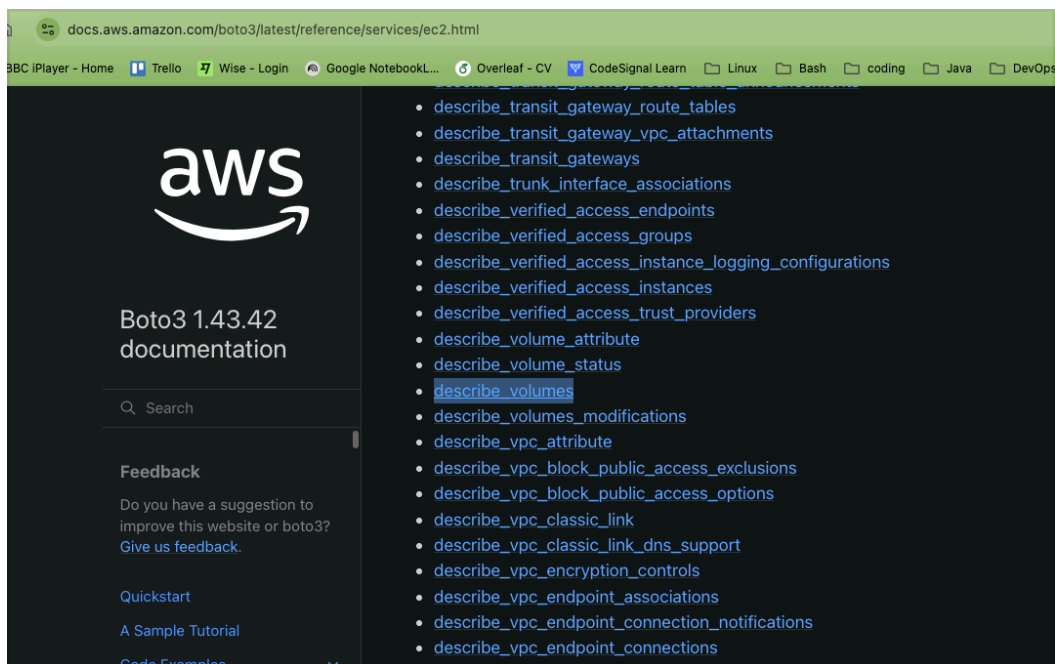


We will write a schedule task that every day collects the volume and creates a snapshot/backup to each volume.

Backup implementation

In AWS documentation we go back to EC2 service and we can find a function called `describe_volumes()`

https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/client/describe_volumes.html



And returns a list of volumes with information like the VolumeId which we will need to create the snapshot later

RETURNS:

Response Syntax

```
{
  'NextToken': 'string',
  'Volumes': [
    {
      'AvailabilityZoneId': 'string',
      'OutpostArn': 'string',
      'SourceVolumeId': 'string',
      'Iops': 123,
      'Tags': [
        {
          'Key': 'string',
          'Value': 'string'
        }
      ],
      'VolumeType': 'standard'|'io1'|'io2'|'gp2'|'sc1'|'st1'|'gp3',
      'FastRestored': True|False,
      'MultiAttachEnabled': True|False,
      'Throughput': 123,
      'SseType': 'sse-ebs'|'sse-kms'|'none',
      'Operator': {
        'Managed': True|False,
        'Principal': 'string',
        'HiddenByDefault': True|False
      },
      'VolumeInitializationRate': 123,
      'VolumeId': 'string',
      'Size': 123
    }
  ]
}
```

The describe_volumes is created on an EC2 client

Request Syntax

```
response = client.describe_volumes(
    VolumeIds=[
        'string',
    ],
    IncludeManagedResources=True|False,
    DryRun=True|False,
```

So here is the beginning of the program:


```
Automation_with_python  Module_14/Automate_Python_chapter-9  volume-backups  Run  volume-backups x
1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-2')
4
5 volumes = ec2_client.describe_volumes()
6 print(volumes) You, Moments ago • Uncommitted changes

Run  volume-backups x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automa
{'Volumes': [{AvailabilityZoneId': 'euw2-az1', 'Iops': 3000, 'VolumeType': 'gp3', 'MultiAttachEnabled
Process finished with exit code 0
```

But we only want the volumes

```
volume-backups.py  describe_volumes_response.py x
1 {
2     'NextToken': 'string',
3     'Volumes': [...]
49 }
```

-> volumes['Volumes']

```
volume-backups.py x
1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-2')
4
5 volumes = ec2_client.describe_volumes()
6 print(volumes['Volumes']) You, Moments ago • Uncommitted changes

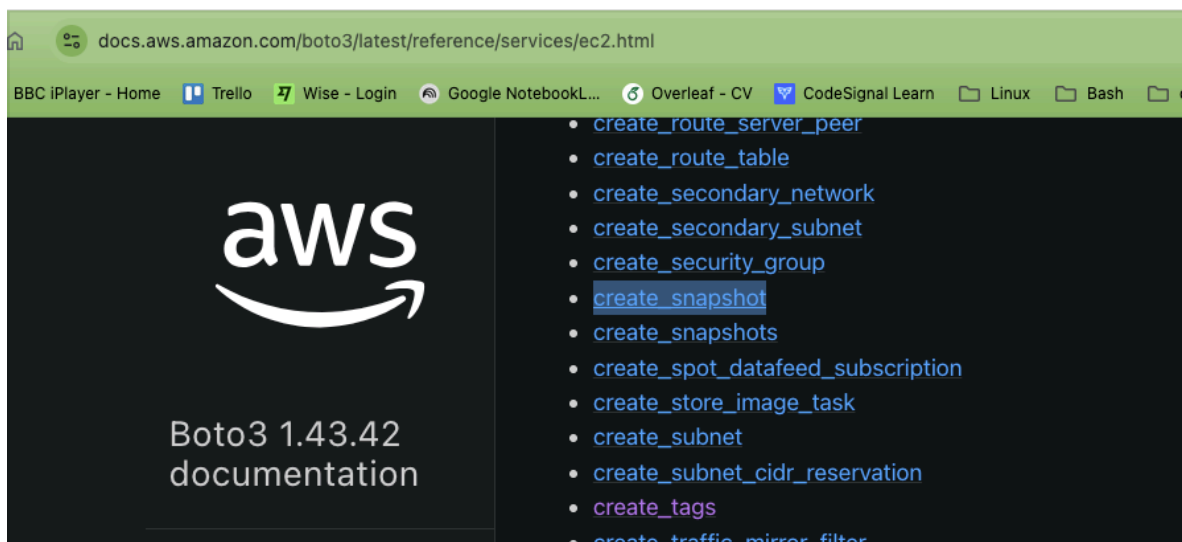
Run  volume-backups x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/Py
[{'AvailabilityZoneId': 'euw2-az1', 'Iops': 3000, 'VolumeType': 'gp3', 'MultiAttachE
Process finished with exit code 0
```

Now we iterate each volume (we have 2 in this demo)

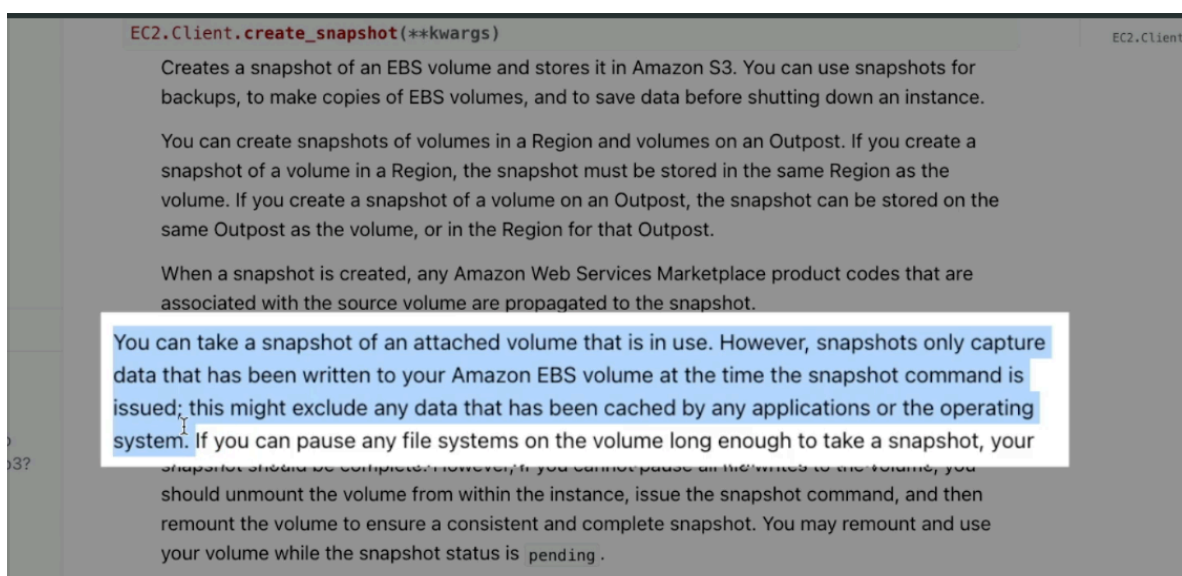
```
volume-backups.py x describe_volumes_response.py
1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-2')
4
5 volumes = ec2_client.describe_volumes()
6 for volume in volumes['Volumes']:
7     ~ You, 2 minutes ago • Uncommitted changes
```

And now we can create a snapshot during the iteration with function `create_snapshot()`

https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/client/create_snapshot.html

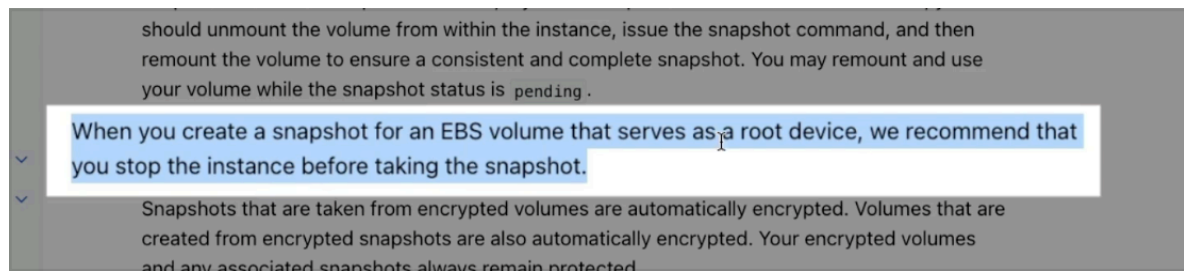


Important notes to do the snapshot: **!!**



If the volume is still in use the snapshot might not include temporarily stored data like application data.

And



If the volume is the root device of an instance then the Instance should be stop before creating the snapshot to make sure data is consistent.

In this DEMO we won't deal with this as it is not part of the scope of the DEMO but it is important for prod environments. **!!**

So we update our code:

A screenshot of a code editor with two tabs: 'volume-backups.py' and 'describe_volumes_response.py'. The 'volume-backups.py' tab is active and shows the following Python code:

```
1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-2')
4
5 volumes = ec2_client.describe_volumes()
6 for volume in volumes['Volumes']:
7     new_snapshot = ec2_client.create_snapshot(
8         VolumeId=volume['VolumeId']
9     )
10 print(new_snapshot)
```

The code is highlighted with syntax coloring. Below the code editor, there is a 'Run' button and a terminal window. The terminal shows the execution output:

```
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/
{'Tags': [], 'SnapshotId': 'snap-0fabf99122caaa4bd', 'VolumeId': 'vol-0cc93cb06d18ae894', 'Sta
{'Tags': [], 'SnapshotId': 'snap-0260d12821e690bac', 'VolumeId': 'vol-0bd237d7116417192', 'Sta
Process finished with exit code 0
```

We got 2 snapshots one per server.

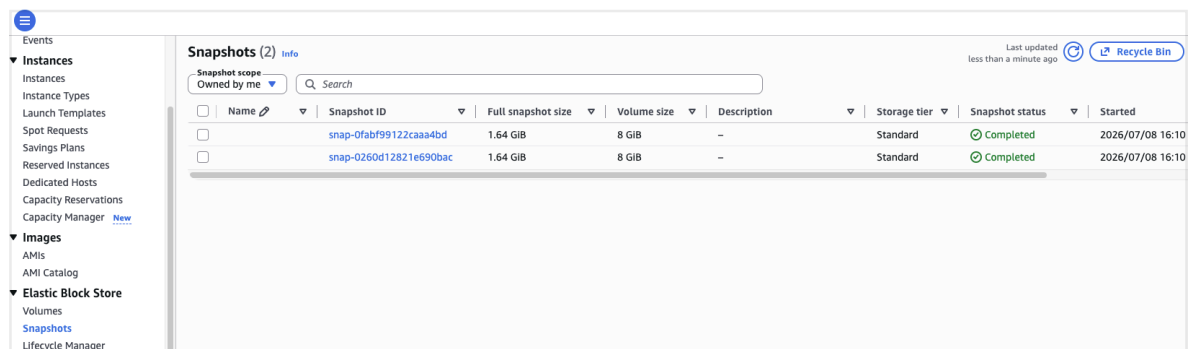
And this is the structure of what we got back per server:

RETURNS:

Response Syntax

```
{
  'OwnerAlias': 'string',
  'OutpostArn': 'string',
  'Tags': [
    {
      'Key': 'string',
      'Value': 'string'
    },
  ],
  'StorageTier': 'archive'|'standard',
  'RestoreExpiryTime': datetime(2015, 1, 1),
  'SseType': 'sse-ebs'|'sse-kms'|'none',
  'AvailabilityZone': 'string',
  'TransferType': 'time-based'|'standard',
  'CompletionDurationMinutes': 123,
  'CompletionTime': datetime(2015, 1, 1),
  'FullSnapshotSizeInBytes': 123,
  'SnapshotId': 'string',
  'VolumeId': 'string',
  'State': 'pending'|'completed'|'error'|'recoverable'|'recovering',
  'StateMessage': 'string',
  'StartTime': datetime(2015, 1, 1),
  'Progress': 'string',
  'OwnerId': 'string',
  'Description': 'string',
  'VolumeSize': 123,
  'Encrypted': True|False,
  'KmsKeyId': 'string',
  'DataEncryptionKeyId': 'string'
}
```

And we check AWS console and we can see the 2 new snapshots



Name	Snapshot ID	Full snapshot size	Volume size	Description	Storage tier	Snapshot status	Started
snap-0fabf99122caa4bd	1.64 GiB	8 GiB	--		Standard	Completed	2026/07/08 16:10
snap-0260d12821e690bac	1.64 GiB	8 GiB	--		Standard	Completed	2026/07/08 16:10

Scheduler implementation

Import the schedule library and set the schedule object to execute the function that contains the creation of the snapshot.

```

1  import boto3
2  import schedule
3
4  ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-2')
5
6  def create_volume_snapshots(): 1 usage new *
7      volumes = ec2_client.describe_volumes()
8      for volume in volumes['Volumes']:
9          new_snapshot = ec2_client.create_snapshot(
10              VolumeId=volume['VolumeId']
11          )
12          print(new_snapshot)
13
14  schedule.every().day.do(create_volume_snapshots)  You, Moments ago • Uncomm.
15
16  # start the scheduler
17  while True:
18      schedule.run_pending()

```

To test the program let's change days to seconds:

```

12  print(new_snapshot)
13
14  schedule.every(5).seconds.do(create_volume_snapshots)
15
16  # start the scheduler
17  while True:
18      schedule.run_pending()  You, A minute ago • Uncommitted changes
19
20

```

Run volume-backups x

```

/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automation
{'Tags': [], 'SnapshotId': 'snap-01784a0ac685ceb1c', 'VolumeId': 'vol-0cc93cb06d18ae894', 'State': 'pending
{'Tags': [], 'SnapshotId': 'snap-09ec1685677fba0c7', 'VolumeId': 'vol-0bd237d7116417192', 'State': 'pending

```

Now we have 4 snapshots, 2 per server

Name	Snapshot ID	Full snapshot size	Volume size	Description	Storage tier	Snapshot status	Started	Progress
☐	snap-01784a0ac685ceb1c	1.64 GiB	8 GiB	--	Standard	Completed	2026/07/08 16:33 GMT+1	100%
☐	snap-0fabf99122caaa4bd	1.64 GiB	8 GiB	--	Standard	Completed	2026/07/08 16:10 GMT+1	100%
☐	snap-0260d12821e690bac	1.64 GiB	8 GiB	--	Standard	Completed	2026/07/08 16:10 GMT+1	100%
☐	snap-09ec1685677fba0c7	1.64 GiB	8 GiB	--	Standard	Completed	2026/07/08 16:33 GMT+1	100%

And an error after that

"botocore.exceptions.ClientError: An error occurred (SnapshotCreationPerVolumeRateExceeded) when calling the CreateSnapshot operation: **The maximum per volume CreateSnapshot request rate has been exceeded.** Use an increasing or variable sleep interval between requests."

```
core/client.py", line 1094, in _make_api_call
```

```
alling the CreateSnapshot operation: The maximum per volume CreateSnapshot request rate has been exceeded. U
```

We tried to create a new snapshot when the previous one was still in creation. So we need more time between triggering the creation of snapshots.

But for our DEMO this is good enough. In reality it will be once a day so there will be no issue regarding timing.

Now let's say we only want to backup production server volumes -> we need to filter the volumes.

So when we get the volumes with `describe_volumes()` we can filter them as per documentation:

Request Syntax

```
response = client.describe_volumes(
    VolumeIds=[
        'string',
    ],
    IncludeManagedResources=True|False,
    DryRun=True|False,
    Filters=[
        {
            'Name': 'string',
            'Values': [
                'string',
            ]
        },
    ],
    NextToken='string',
    MaxResults=123
)
```

And these are the values that the filter can use, one of which is 'tag:<key>'

• Filters (list) –

The filters.

- `attachment.attach-time` - The time stamp when the attachment initiated.
- `attachment.delete-on-termination` - Whether the volume is deleted on instance termination.
- `attachment.device` - The device name specified in the block device mapping (for example, `/dev/sda1`).
- `attachment.instance-id` - The ID of the instance the volume is attached to.
- `attachment.status` - The attachment state (`attaching` | `attached` | `detaching`).
- `availability-zone` - The Availability Zone in which the volume was created.
- `availability-zone-id` - The ID of the Availability Zone in which the volume was created.
- `create-time` - The time stamp when the volume was created.
- `encrypted` - Indicates whether the volume is encrypted (`true` | `false`)
- `fast-restored` - Indicates whether the volume was created from a snapshot that is enabled for fast snapshot restore (`true` | `false`).
- `multi-attach-enabled` - Indicates whether the volume is enabled for Multi-Attach (`true` | `false`)
- `operator.managed` - A Boolean that indicates whether this is a managed volume.
- `operator.principal` - The principal that manages the volume. Only valid for managed volumes, where `managed` is `true`.
- `size` - The size of the volume, in GiB.
- `snapshot-id` - The snapshot from which the volume was created.
- `status` - The state of the volume (`creating` | `available` | `in-use` | `deleting` | `deleted` | `error`).
- `tag :<key>` - The key/value combination of a tag assigned to the resource. Use the tag key in the filter name and the tag value as the filter value. For example, to find all resources that have a tag with the key `Owner` and the value `TeamA`, specify `tag:Owner` for the filter name

So we need our volumes to have tags to be able to use this filter. The best way is automatically assign tags to the volumes with a python script but for this Demo we will add the tag to the prod volume manually:

So I select the prod instance and then in the storage tab I select the volume

The screenshot shows the AWS Management Console interface. At the top, there's a header for 'Instances (1/2)' with a search bar and filter options. Below this, a table lists instances. The 'prod' instance (ID: i-08cf2b587fe1ccd0b) is selected. The 'Storage' tab is active, showing the 'Block devices' section. A table lists the attached volumes. The volume 'vol-0bd237d7116417192' is highlighted.

Volume ID	Device name	Volume size (GiB)	Volume State	Attachment status	EBS card index	Attachment time
vol-0bd237d7116417192	/dev/xvda	8	In-use	Attached	0	2026/07/08 15:39 GMT+1

Once I am inside the volume section I go to the tags tab and add the tag:

Volumes (1/1) Info

Saved filter sets

Choose filter set

Search

Volume ID = vol-0bd237d7116417192

Clear filters

Name

Volume ID

Type

Size

IOPS

Throughput

Snapshot ID

Source volume ID

vol-0bd237d7116417192

gp3

8 GiB

3000

125

snap-0dbdeff2...

-

Volume ID: vol-0bd237d7116417192

Details

Status checks

Monitoring

Tags

Tags

Filter tags

Key

Value

The selected resource currently has no tags.

Manage tags

EC2 > Volumes > vol-0bd237d7116417192 > Manage tags

Manage tags Info

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name

Value - optional

Q prod

Remove

Add tag

You can add 49 more tags.

Cancel

Save

And it has the tag now:

Successfully updated tags for volume vol-0bd237d7116417192.

Volumes (1/1) [Info](#)

Saved filter sets
Choose filter set Search

Volume ID = vol-0bd237d7116417192

☒	Name ✎	Volume ID	Type	Size	IOPS	Throughput	Snapshot ID	Source volume
☒	prod	vol-0bd237d7116417192	gp3	8 GiB	3000	125	snap-0dbdeff2...	-

Volume ID: vol-0bd237d7116417192 (prod)

Details | Status checks | Monitoring | **Tags**

Tags

Filter tags

Key	Value
Name	prod

And we have one volume with a tag and another without tag

Volumes (2) [Info](#)

Saved filter sets
Choose filter set Search

☐	Name ✎	Volume ID	Type	Size	IOPS	Throughput	Snapshot ID	Source volume
☐		vol-0cc93cb06d18ae894	gp3	8 GiB	3000	125	snap-0dbdeff2...	-
☐	prod	vol-0bd237d7116417192	gp3	8 GiB	3000	125	snap-0dbdeff2...	-

Next we can filter in our python program

```

6      def create_volume_snapshots(): 1 usage new *
7          volumes = ec2_client.describe_volumes(
8              Filters=[
9                  {
10                     'Name': ' ',
11                     'Values': [' ']}
12             ]
13         )
14
15     for volume in volumes['Volumes']: You, A minute ago

```

using tag with key 'Name' and value 'prod'

```

7         volumes = ec2_client.describe_volumes(
8             Filters=[
9                 {
10                     'Name': 'tag:Name',
11                     'Values': ['prod']
12                 }
13             ]
14         )

```

And if we were to get prod and staging environments we can do this:

```

7     def create_volume_snapshots():
8         volumes = ec2_client.describe_volumes(
9             Filters=[
10                 {
11                     'Name': 'tag:Name',
12                     'Values': ['prod', 'staging']
13                 }
14             ]
15         )

```

And the tag key could be 'Environment' instead of 'Name'

Let's try again but using 20 seconds to avoid the timing error:

```

18         )
19         print(new_snapshot)
20
21     schedule.every(20).seconds.do(create_volume_snapshots)
22
23     # start the scheduler
24     while True:
25         schedule.run_pending()
26
27

```

Run volume-backups x

```

/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automat
{'Tags': [], 'SnapshotId': 'snap-05d7d1176f5c40753', 'VolumeId': 'vol-0bd237d7116417192', 'State': 'per

```

